

Lab 3: Architecture Selection & Justification

Inter-City Freight Load Matching Marketplace (Problem Statement #28)

Architecture Selection: We chose Microservices Architecture for the Inter-City Freight Load Matching Marketplace System.

Architectural Choice

Microservices. The marketplace is split into independently deployable components behind an API Gateway: Carrier Verification, Load & Bidding, Proof-of-Delivery, Escrow Payment and Notification services, plus the Web Portal client. They interact only through the provided/required interfaces in the component diagram (synchronous REST over TLS 1.3, and AMQP messaging for alerts).

Two Reasons for This Choice

1. **Uneven, bursty workload needs independent scaling.** Bidding spikes when auctions close, while permit verification and proof-of-delivery uploads are low-volume. Separating Load & Bidding lets us scale only that service, and keeps heavy bidding traffic from slowing escrow release (NFR-001: payment within 60 seconds).
2. **Fault isolation across distinct business domains.** Shippers, carriers and administrators use different capabilities (bidding, verification, delivery, payments). If Notification or Proof-of-Delivery fails or is redeployed, bidding keeps running, and the financially sensitive Escrow Payment service can be updated, audited and deployed on its own.

Security Advantage

The API Gateway is the single public entry point. It terminates TLS 1.3, authenticates users and enforces role-based access (Shipper, Carrier, Administrator) before any request reaches a service. Internal services are not publicly exposed, talk to each other over mutual TLS, and keep data encrypted (AES-256) in their own datastores. A compromise of one service, such as Notification, therefore cannot expose escrow or credential data, which directly supports NFR-002.

Performance Benefit

Notifications are sent through an asynchronous message queue, so a bid alert never blocks the carrier's request. The dedicated, horizontally scalable Escrow Payment service is triggered directly by Proof-of-Delivery, so milestone funds can be released inside the 60-second target even during peak bidding load.

Alternatives Considered and Trade-offs

Layered architecture would bundle bidding, payments and verification into one deployable unit that is hard to scale selectively. Client-Server would centralise everything on one server, creating a single point of failure and a bottleneck at auction peaks. The price of microservices is extra operational complexity, network latency and data-consistency work; we accept this and handle cross-service steps (Bidding, Proof-of-Delivery, Escrow) with an event-driven, saga-style flow.